UNIT-5
OOAD Object Modeling Techniques

The object modeling techniques is an methodology of object oriented analysis, design and implementation that focuses on creating a model of objects from the real world and then to use this model to develop object–oriented software. object modeling technique, OMT was developed by James Rambaugh. Now-a-days, OMT is one of the most popular object oriented development techniques. It is primarily used by system and software developers to support full life cycle development while targeting object oriented implementations.

OMT has proven itself easy to understand, to draw and to use. It is very successful in many application domains: telecommunication, transportation, compilers etc. The popular object modeling technique are used in many real world problems. The object-oriented paradigm using the OMT spans the entire development cycle, so there is no need to transform one type of model to another.

Phase of OMT

The OMT methodology covers the full software development life cycle. The methodology has the following phase.

1. **Analysis** - Analysis is the first phase of OMT methodology. The aim of analysis phase is to build a model of the real world situation to show its important properties and domain. This phase is concerned with preparation of precise and correct modelling of the real world. The analysis phase starts with defining a problem statement which includes a set of goals. This problem statement is then expanded into three models; an object model, a dynamic model and a functional model. The object model shows the static data structure or skeleton of the real world system and divides the whole application into objects. In others words, this model represents the artifacts of the system. The dynamic model represents the interaction between artifacts above designed represented as events, states and transitions. The functional model represents the methods of the system from the data flow perspective. The analysis phase generates object model diagrams, state diagrams, event flow diagrams and data flow diagrams.

2. **System design** - The system design phase comes after the analysis phase. System design phase determines the overall system architecture using subsystems, concurrent tasks and data storage. During system design, the high level structure of the system is designed. The decisions made during system design are:

    - The system is organized in to sub-systems which are then allocated to processes and tasks, taking into account concurrency and collaboration.
    - Persistent data storage is established along with a strategy to manage shared or global information.
    - Boundary situations are checked to help guide trade off priorities.

3. **Object design** - The object design phase comes after the system design phase is over. Here the implementation plan is developed. Object design is concerned with fully classifying the existing and remaining classes, associations, attributes and operations necessary for implementing a solution to the problem. In object design:

- Operations and data structures are fully defined along with any internal objects needed for implementation.
- Class level associations are determined.
- Issues of inheritance, aggregation, association and default values are checked.

4. **Implementation** - Implementation pahse of the OMT is a matter of translating the design in to a programming language constructs. It is important to have good software engineering practice so that the design phase is smoothly translated in to the implementation phase. Thus while selecting programming language all constructs should be kept in mind for following noteworthy points.
    - To increase flexibility.
    - To make amendments easily.
    - For the design traceability.
    - To increase efficiency.

Relationship with other OMT techniques

5.1 Overview of OMT

As mentioned before, OMT comprises of three models:

- The object model represents the static, structural, 'data' aspects of a system;
- The dynamic model represents the temporal, behavorial, 'control' aspects of a system;
- The functional model represents the transformational, 'functional' aspects of a system.

The three kinds of models separate a system into orthogonal views, they all describe one aspect of the system, but contain references to the other models.

The object model provides the data essential framework into which the dynamic and functional models can be placed, it defines the data stuctue that the other models operate on. In the end the three models come together in implementation, which involves data (object model), sequencing (dynamic model) and operations (functional model).

Below we will discuss the dynamic and functional model. After that we will show the relationships between the three models.

5.2 Dynamic model

Those aspects of a system that are concerned with time and changes are captured in the dynamic model. It defines the control structure: the aspects of a system that

describes the sequences of operations that occur in response to external stimuli, without consideration of what the operations do (functional model), what they operate on (object model), or how they are implemented.
The major dynamic modelling concepts are events (external stimuli), and states (values and links of an object). The pattern of events, states, and state transitions for a given object class, can be represented as a state diagram. The dynamic model consists of multiple state diagrams, one for each object class with important dynamic behavior.

5.3 Functional model

The functional model shows the computation and functional derivation of the data values in it, without indication how (implementation), when (dynamic model), or why the values are computed. The relationshiop between values in a computation are showed in a data flow diagram. It consists of:

processes

transformation of data;

actors

procucers and consumers of values;

data stores

creators of delay between creation and use of data;

data flows

relation between processen, actors and data stores.

5.4 Relationships between the three models

5.4.1 Object and dynamic model

The dynamic model specifies allowable sequences of changes to objects from the object model. The states of the dynamic model can be related to classes of attibute and links values of an object, events can be represented as operations on the object model.

The object model concepts of generalization, aggregation and inheritance also apply to

the dynamic model.

generalization

Every substate of an object restricts the values that an object can have, this is a special form of generalization, namely generalization by restriction. It is clear that both generalization on classes and on states partition the set of possible object values. When there exist inherent differences between objects, they are modeled as different classen. When the differences are temporary, they are modeled as different states of the same class.

aggregation

A composite state is the aggregation of more than one substate. Aggregation exists at two levels, aggreation of objects (the composite state is an assembly of the individual states of the parts) and aggregation in objects (groups of attribute and links values define the composite state;

inheritance

It is possible to draw hierarchies of events and of states. Events inherit attibutes (the data values) of their parent(s) and states inherit the possible transitions. The dynamic model of a class is also inherited by its subclasses.

5.4.2 Object and functional model

All four components of the functional model can be related to the object model.

processes

The functional model shows the processes that have to be implemented in the methods of the objects. The processes in the functional model shows the objects that are related by function. Often one in- or output is the target (client) of a process. The other inputs are the parameters (suppliers). These client-supplier relationships establish implementation dependencies between the related classes.

actors

These are the objects in the object model.

data stores

These are also objects in the object model or attributes of objects

data flows

These are values in the object model. Data flows to or from actors represent operations on or by objects. Data flows to or from data stores represent queries or updates.

5.4.3 Dynamic and functional model

The relationship between these two models is that the dynamic model states when operations are perfomed, and the functional model states how they are performed and which arguments are needed. There is however a difference between operations on actors and operations on data stores. Because actors are active objects, the dynamic model has to specify when it acts. The data stores are passive objects, they only respond to updates and queries, therefore you do not have to specify in the dynamic model when they act.

In OOP the various properties of an object are analyzed. Understanding the properties assists in developing efficient modules. These properties include:

1. State—the properties of the object that cause a specific reaction to any specific event. For example, a terminal has several states: off; on and running; on and idle; and under maintenance.
2. Behavior—the way an object reacts to interactions from and with other objects. Behavior is determined by the set of operations defined as functional for that object. For example, an invalid entry from the person object is reacted to by an error message by the application package object.
3. Identity—objects are unique, merely because they exist separately. Objects are not grouped with other objects because of similar behavior. Nor can any object have more than one identity or name.
4. Encapsulation—this is where data and functionality are combined. Data and corresponding programs are elements that are isolated together. This prevents the corruption of their internal state.
5. Messaging—objects send messages to each other in order to access each other and to request a response. The object responds back to the requestor based on its defined behavior.
6. Collaboration—objects use each other to accomplish tasks. They share the responsibility for task completion.

7. Information hiding—this is where the activities within the object are hidden from other objects on the outside. This prevents corruption of the object or interference between objects.
8. Inheritance—this is a process of building objects by combining other objects together. By linking one object to another, one often inherits the properties of the other.

The benefits of OOP include:

- OOP offers usability and reusability of programming code without complete overhaul and replacement.
- OOP has significantly increased the speed of the programming process, both in development and in maintenance.
- OOP offers easier understanding of programming code for co-developers and outsiders trying to understand the code.
- OOP makes it easier to break down complex systems into components and then allows for the prototyping of these systems.
- OOP offers simplified implementation in that basic objects can be independently debugged and do not require as much testing.
- OOP is flexible and highly adaptable to changing business requirements

Definition and Overview of ODBMS

The **ODBMS** which is an abbreviation for **object oriented database management system**, is the data model in which data is stored in form of objects, which are instances of classes. These classes and objects together makes an object oriented data model.

Components of Object Oriented Data Model:

The OODBMS is based on three major components, namely: Object structure, Object classes, and Object identity. These are explained as following below.

1. Object Structure:

The structure of an object refers to the properties that an object is made up of. These properties of an object are referred to as an attribute. Thus, an object is a real world world entity with certain attributes that makes up the object structure. Also an object encapsulates the data code into a single unit which in turn provides data abstraction by hiding the implementation details from the user.

The object structure is further composed of three types of components: Messages,

Methods, and Variables. These are explained as following below.

1. **Messages –**
   A message provides an interface or acts as a communication medium between an object and the outside world. A message can be of two types:
   - **Read-only message:** If the invoked method does not change the value of a variable, then the invoking message is said to be a read-only message.
   - **Update message:** If the invoked method changes the value of a variable, then the invoking message is said to be an update message.
2. **Methods –**
   When a message is passed then the body of code that is executed is known as a method. Every time when a method is executed, it returns a value as output. A method can be of two types:
   - **Read-only method:** When the value of a variable is not affected by a method, then it is known as read-only method.
   - **Update-method:** When the value of a variable changes by a method, then it is known as an update method.
3. **Variables –**
   It stores the data of an object. The data stored in the variables makes the object distinguishable from one another.

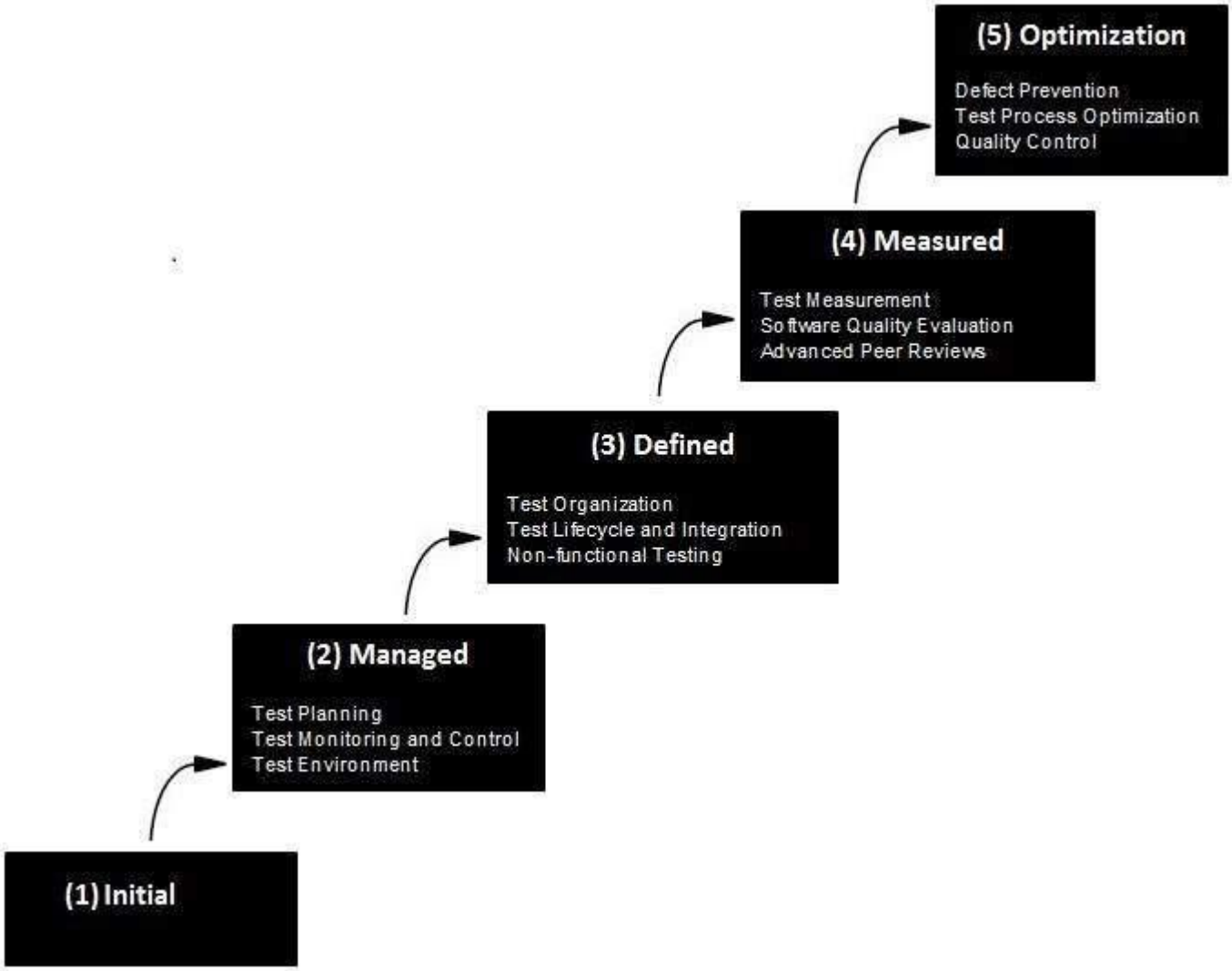
2. Object Classes:

An object which is a real world entity is an instance of a class. Hence first we need to define a class and then the objects are made which differ in the values they store but share the same class definition. The objects in turn corresponds to various messages and variables stored in it

What is Capability Maturity Model?

The Software Engineering Institute (SEI) Capability Maturity Model (CMM) specifies an increasing series of levels of a software development organization. The higher the level, the better the software development process, hence reaching each level is an expensive and time-consuming process.

Levels of CMM



- **Level One :Initial** - The software process is characterized as inconsistent, and occasionally even chaotic. Defined processes and standard practices that exist are abandoned during a crisis. Success of the organization majorly depends on an individual effort, talent, and heroics. The heroes eventually move on to other organizations taking their wealth of knowledge or lessons learnt with them.
- **Level Two: Repeatable** - This level of Software Development Organization has a basic and consistent project management processes to track cost, schedule, and functionality. The process is in place to repeat the earlier successes on projects with similar applications. Program management is a key characteristic of a level two organization.

- **Level Three: Defined** - The software process for both management and engineering activities are documented, standardized, and integrated into a standard software process for the entire organization and all projects across the organization use an approved, tailored version of the organization's standard software process for developing,testing and maintaining the application.
- **Level Four: Managed** - Management can effectively control the software development effort using precise measurements. At this level, organization set a quantitative quality goal for both software process and software maintenance. At this maturity level, the performance of processes is controlled using statistical and other quantitative techniques, and is quantitatively predictable.
- **Level Five: Optimizing** - The Key characteristic of this level is focusing on continually improving process performance through both incremental and innovative technological improvements. At this level, changes to the process are to improve the process performance and at the same time maintaining statistical probability to achieve the established quantitative process-improvement objectives

ISO:-

ISO 9000 is defined as a set of international standards on quality management and quality assurance developed to help companies effectively document the quality system elements needed to maintain an efficient quality system. They are not specific to any one industry and can be applied to organizations of any size.

ISO 9000 can help a company satisfy its customers, meet regulatory requirements, and achieve continual improvement. It should be considered to be a first step or the base level of a quality system.

- ISO 9000 vs. 9001
- 30 years of ISO 9000
- ISO 9000 resources

ISO 9000 VS. 9001

ISO 9000 is a series, or family, of quality management standards, while ISO 9001 is a standard within the family. The ISO 9000 family of standards also contains an individual standard named ISO 9000. This standard lays out the fundamentals and vocabulary for quality management systems (QMS).

ISO 9000 series of Standards

The ISO 9000 family contains these standards:

- ISO 9001:2015: Quality Management Systems - Requirements
- ISO 9000:2015: Quality Management Systems - Fundamentals and Vocabulary (definitions)
- ISO 9004:2018: Quality Management - Quality of an Organization - Guidance to Achieve Sustained Success (continuous improvement)
- ISO 19011:2018: Guidelines for Auditing Management Systems

ASQ is the only place where organizations can obtain the American National Standard Institute (ANSI) versions of these standards in the ISO 9000 family.

ISO 9000 history and revisions: ISO 9000:2000, 2008, and 2015

ISO 9000 was first published in 1987 by the International Organization for Standardization (ISO), a specialized international agency for standardization composed of the national standards bodies of more than 160 countries. The standards underwent major revisions in 2000 and 2008. The most recent versions of the standard, ISO 9000:2015 and ISO 9001:2015 , were published in September 2015.

ASQ administers the U.S. Technical Advisory Groups and subcommittees that are responsible for developing the ISO 9000 family of standards. In its standards development work, ASQ is accredited by ANSI.

ISO 9000:2000

ISO 9000:2000 refers to the ISO 9000 update released in the year 2000.

The ISO 9000:2000 revision had five goals:

1. Meet stakeholder needs
2. Be usable by all sizes of organizations
3. Be usable by all sectors
4. Be simple and clearly understood
5. Connect quality management system to business processes

ISO 9000:2000 was again updated in 2008 and 2015. ISO 9000:2015 is the most current version.

ISO 9000:2015 principles of Quality Management

The ISO 9000:2015 and ISO 9001:2015 standards are based on seven quality management principles that senior management can apply to promote organizational improvement.

ISO 9000 Quality Management Principles

COCOMO Model

Boehm proposed COCOMO (Constructive Cost Estimation Model) in 1981.COCOMO is one of the most generally used software estimation models in the world. COCOMO predicts the efforts and schedule of a software product based on the size of the software.

The necessary steps in this model are:

1. Get an initial estimate of the development effort from evaluation of thousands of delivered lines of source code (KDLOC).
2. Determine a set of 15 multiplying factors from various attributes of the project.
3. Calculate the effort estimate by multiplying the initial estimate with all the multiplying factors i.e., multiply the values in step1 and step2.

The initial estimate (also called nominal estimate) is determined by an equation of the form used in the static single variable models, using KDLOC as the measure of the size. To determine the initial effort E_i in person-months the equation used is of the type is shown below

E_i=a*(KDLOC)b

The value of the constant a and b are depends on the project type.

In COCOMO, projects are categorized into three types:

1. Organic
2. Semidetached
3. Embedded

1. Organic: A development project can be treated of the organic type, if the project deals with developing a well-understood application program, the size of the development team is reasonably small, and the team members are experienced in developing similar methods of projects. **Examples of this type of projects are simple business systems, simple inventory management systems, and data processing systems.**

2. Semidetached: A development project can be treated with semidetached type if the development consists of a mixture of experienced and inexperienced staff. Team members may have finite experience in related systems but may be unfamiliar with some aspects of the order being developed. **Example of Semidetached system includes developing a new operating system (OS), a Database Management System (DBMS), and complex inventory management system.**

3. Embedded: A development project is treated to be of an embedded type, if the software being developed is strongly coupled to complex hardware, or if the stringent regulations on the operational method exist. **For Example:** ATM, Air Traffic control.

For three product categories, Bohem provides a different set of expression to predict effort (in a unit of person month)and development time from the size of estimation in KLOC(Kilo Line of code) efforts estimation takes into account the productivity loss due to holidays, weekly off, coffee breaks, etc.

According to Boehm, software cost estimation should be done through three stages:

1. Basic Model

2. Intermediate Model
3. Detailed Model

1. Basic COCOMO Model: The basic COCOMO model provide an accurate size of the project parameters. The following expressions give the basic COCOMO estimation model:

$$\textbf{Effort}=a_1*(\textbf{KLOC})\ a_2\ \textbf{PM}$$
$$\textbf{Tdev}=b_1*(\textbf{efforts})b_2\ \textbf{Months}$$

Where

KLOC is the estimated size of the software product indicate in Kilo Lines of Code,

a_1,a_2,b_1,b_2 are constants for each group of software products,

Tdev is the estimated time to develop the software, expressed in months,

Effort is the total effort required to develop the software product, expressed in **person months (PMs)**.

Estimation of development effort

For the three classes of software products, the formulas for estimating the effort based on the code size are shown below:

Organic: Effort = 2.4(KLOC) 1.05 PM

Semi-detached: Effort = 3.0(KLOC) 1.12 PM

Embedded: Effort = 3.6(KLOC) 1.20 PM

Estimation of development time

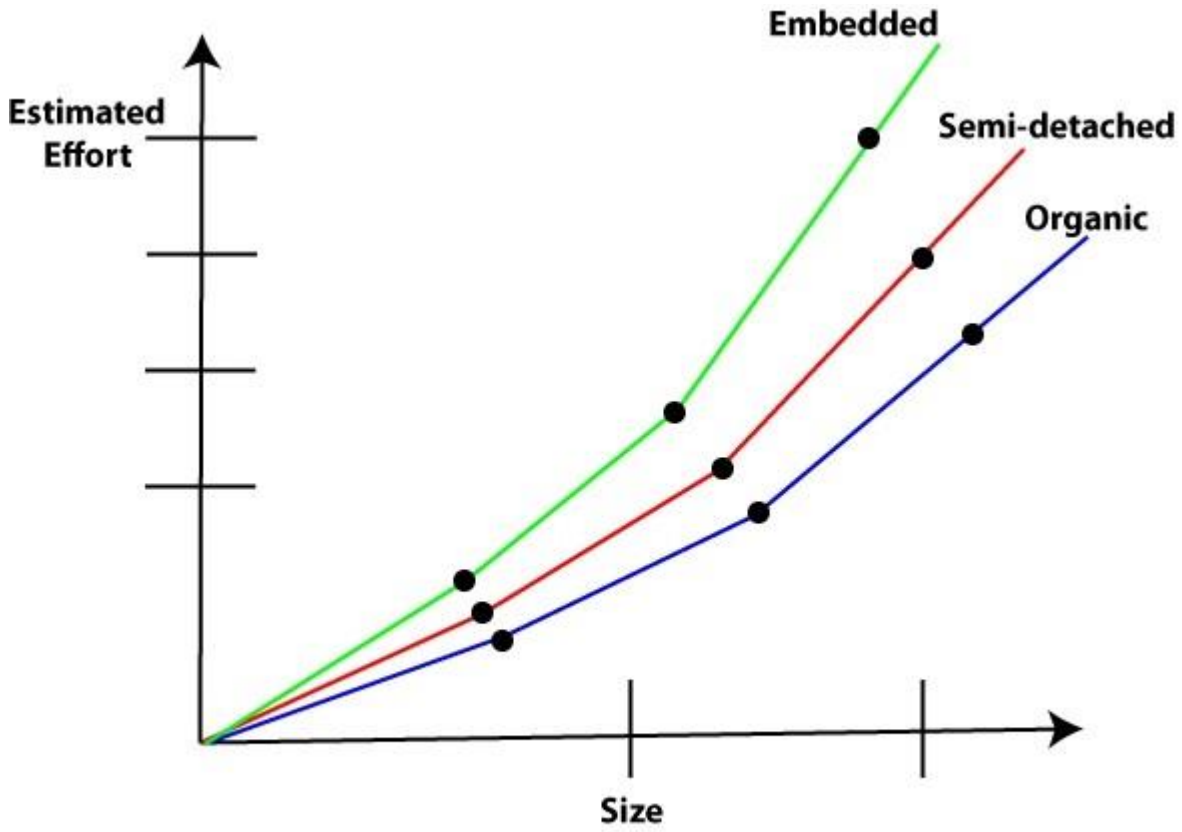
For the three classes of software products, the formulas for estimating the development time based on the effort are given below:

Organic: Tdev = 2.5(Effort) 0.38 Months

Semi-detached: Tdev = 2.5(Effort) 0.35 Months

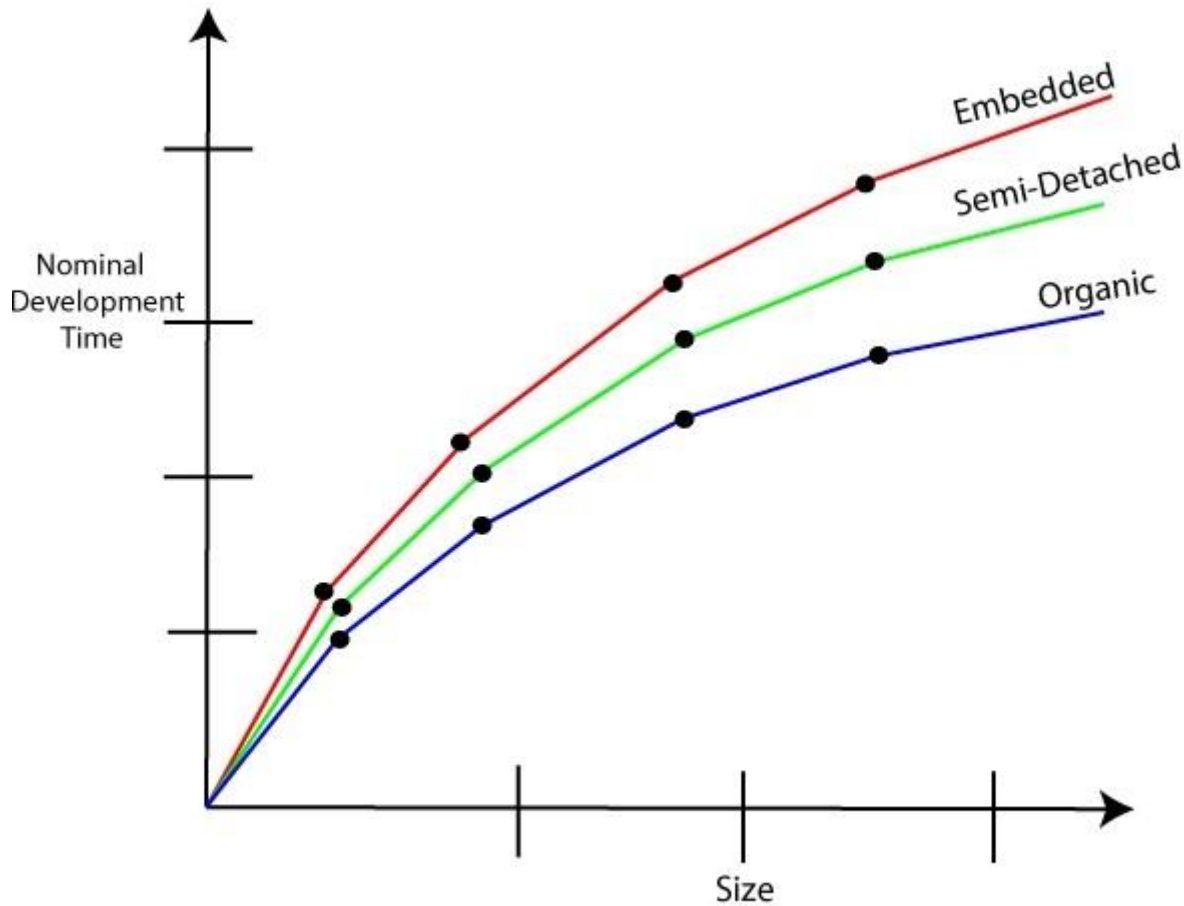
Embedded: Tdev = 2.5(Effort) 0.32 Months

Some insight into the basic COCOMO model can be obtained by plotting the estimated characteristics for different software sizes. Fig shows a plot of estimated effort versus product size. From fig, we can observe that the effort is somewhat superliner in the size of the software product. Thus, the effort required to develop a product increases very rapidly with project size.



Effort versus product size

The development time versus the product size in KLOC is plotted in fig. From fig it can be observed that the development time is a sub linear function of the size of the product, i.e. when the size of the product increases by two times, the time to develop the product does not double but rises moderately. This can be explained by the fact that for larger products, a larger number of activities which can be carried out concurrently can be identified. The parallel activities can be carried out simultaneously by the engineers. This reduces the time to complete the project. Further, from fig, it can be observed that the development time is roughly the same for all three categories of products. For example, a 60 KLOC program can be developed in approximately 18 months, regardless of whether it is of organic, semidetached, or embedded type.



Development time versus size

From the effort estimation, the project cost can be obtained by multiplying the required effort by the manpower cost per month. But, implicit in this project cost computation is the assumption that the entire project cost is incurred on account of the manpower cost alone. In addition to manpower cost, a project would incur costs due to hardware and software required for the project and the company overheads for administration, office space, etc.

It is important to note that the effort and the duration estimations obtained using the COCOMO model are called a nominal effort estimate and nominal duration estimate. The term nominal implies that if anyone tries to complete the project in a time shorter than the estimated duration, then the cost will increase drastically. But, if anyone completes the project over a longer period of time than the estimated, then there is almost no decrease in the estimated cost value.

Example1: Suppose a project was estimated to be 400 KLOC. Calculate the effort and development time for each of the three model i.e., organic, semi-detached & embedded.

Solution: The basic COCOMO equation takes the form:

Effort=a_1*(KLOC) a_2 PM
Tdev=b_1*(efforts)b_2 Months
Estimated Size of project= 400 KLOC

(i) Organic Mode

E = 2.4 * (400)1.05 = 1295.31 PM
D = 2.5 * (1295.31)0.38=38.07 PM

(ii) Semidetached Mode

E = 3.0 * (400)1.12=2462.79 PM
D = 2.5 * (2462.79)0.35=38.45 PM

(iii) Embedded Mode

E = 3.6 * (400)1.20 = 4772.81 PM
D = 2.5 * (4772.8)0.32 = 38 PM

Example2: A project size of 200 KLOC is to be developed. Software development team has average experience on similar type of projects. The project schedule is not very tight. Calculate the Effort, development time, average staff size, and productivity of the project.

Solution: The semidetached mode is the most appropriate mode, keeping in view the size, schedule and experience of development time.

Hence E=3.0(200)1.12=1133.12PM
D=2.5(1133.12)0.35=29.3PM

$$\text{Average Staff Size (SS)} = \frac{E}{D}\ \text{Persons}$$

$$= \frac{1133.12}{29.3} = 38.67\ \text{Persons}$$

$$\text{Productivity} = \frac{\text{KLOC}}{E} = \frac{200}{1133.12} = 0.1765\ \text{KLOC/PM}$$

P = 176 LOC/PM

2. Intermediate Model: The basic Cocomo model considers that the effort is only a function of the number of lines of code and some constants calculated according to the various software systems. The intermediate COCOMO model recognizes these facts and refines the initial estimates obtained through the basic COCOMO model by using a set of 15 cost drivers based on various attributes of software engineering.

Classification of Cost Drivers and their attributes:

(i) Product attributes -

- Required software reliability extent
- Size of the application database
- The complexity of the product

Hardware attributes -

- Run-time performance constraints
- Memory constraints
- The volatility of the virtual machine environment
- Required turnabout time

Personnel attributes -

- Analyst capability

- Software engineering capability
- Applications experience
- Virtual machine experience
- Programming language experience

Project attributes -

- Use of software tools
- Application of software engineering methods
- Required development schedule

The cost drivers are divided into four categories:

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very High	Extra High
Product Attributes						
RELY	0.75	0.88	1.00	1.15	1.40	..
DATA	..	0.94	1.00	1.08	1.16	..
CPLX	0.70	0.85	1.00	1.15	1.30	1.65
Computer Attributes						
TIME	..	..	1.00	1.11	1.30	1.66
STOR	..	..	1.00	1.06	1.21	1.56
VIRT	..	0.87	1.00	1.15	1.30	..
TURN	..	0.87	1.00	1.07	1.15	..

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Personnel Attributes						
ACAP	1.46	1.19	1.00	0.86	0.71	..
AEXP	1.29	1.13	1.00	0.91	0.82	..
PCAP	1.42	1.17	1.00	0.86	0.70	..
VEXP	1.21	1.10	1.00	0.90	..	..
LEXP	1.14	1.07	1.00	0.95	..	..
Project Attributes						
MODP	1.24	1.10	1.00	0.91	0.82	..
TOOL	1.24	1.10	1.00	0.91	0.83	..
SCED	1.23	1.08	1.00	1.04	1.10	..

Intermediate COCOMO equation:

$$E=a_i\,(KLOC)\,b_i*EAF$$
$$D=c_i\,(E)d_i$$

Coefficients for intermediate COCOMO

Project	a_i	b_i	c_i	d_i
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

3. Detailed COCOMO Model: Detailed COCOMO incorporates all qualities of the standard version with an assessment of the cost driver?s effect on each method of the software engineering process. The detailed model uses various effort multipliers for each cost driver property. In detailed cocomo, the whole software is differentiated into multiple modules, and then we apply COCOMO in various modules to estimate effort and then sum the effort.

The Six phases of detailed COCOMO are:

1. Planning and requirements
2. System structure
3. Complete structure
4. Module code and test
5. Integration and test
6. Cost Constructive model

The effort is determined as a function of program estimate, and a set of cost drivers are given according to every phase of the software lifecycle.

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very High	Extra High
Product Attributes						
RELY	0.75	0.88	1.00	1.15	1.40	..
DATA	..	0.94	1.00	1.08	1.16	..
CPLX	0.70	0.85	1.00	1.15	1.30	1.65
Computer Attributes						
TIME	..	..	1.00	1.11	1.30	1.66
STOR	..	..	1.00	1.06	1.21	1.56
VIRT	..	0.87	1.00	1.15	1.30	..
TURN	..	0.87	1.00	1.07	1.15	..

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Personnel Attributes						
ACAP	1.46	1.19	1.00	0.86	0.71	..
AEXP	1.29	1.13	1.00	0.91	0.82	..
PCAP	1.42	1.17	1.00	0.86	0.70	..
VEXP	1.21	1.10	1.00	0.90	..	..
LEXP	1.14	1.07	1.00	0.95	..	..
Project Attributes						
MODP	1.24	1.10	1.00	0.91	0.82	..
TOOL	1.24	1.10	1.00	0.91	0.83	..
SCED	1.23	1.08	1.00	1.04	1.10	..

Intermediate COCOMO equation:

$$E=a_i\ (KLOC)\ b_i*EAF$$
$$D=c_i\ (E)d_i$$

Coefficients for intermediate COCOMO

Project	a_i	b_i	c_i	d_i
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

3. Detailed COCOMO Model: Detailed COCOMO incorporates all qualities of the standard version with an assessment of the cost driver?s effect on each method of the software engineering process. The detailed model uses various effort multipliers for each cost driver property. In detailed cocomo, the whole software is differentiated into multiple modules,

and then we apply COCOMO in various modules to estimate effort and then sum the effort.

The Six phases of detailed COCOMO are:

1. Planning and requirements
2. System structure
3. Complete structure
4. Module code and test
5. Integration and test
6. Cost Constructive model

The effort is determined as a function of program estimate, and a set of cost drivers are given according to every phase of the software lifecycle.